

Issues We Hit, and How We Fixed Them

BTC Volatility Prediction — engineering issue log and postmortem

Mohammed Hejairi · Cohort DSB2 · 2026-08-29

Purpose

This document is a chronological record of every technical issue, bug, and analytical flaw encountered while building the Bitcoin volatility prediction capstone, and exactly how each one was diagnosed and resolved. It exists as an engineering record separate from the main report: the main report presents the finished analysis, while this document shows the debugging trail behind it — what looked wrong, why it was wrong, and what changed.

None of the issues below were cosmetic. Several directly determined which numbers ended up in the final report — most notably the ATR scale fix (Issue 5) and the forecast-horizon redesign (Issue 7), which is why they are documented in full rather than folded quietly into the final notebook.

Issues are numbered in the order they were found and keep those numbers permanently as identifiers; the table and section order below group them by which stage of the pipeline they actually belong to (Data Collection, EDA, Feature Engineering, Modelling, Live Testing, Report/Deck Generation) rather than by discovery order.

Summary Table

#	Issue	Pipeline Stage	Resolution
1	nbconvert working-directory mismatch	Data Collection / Ingestion	Symlinked <code>notebooks/data</code> and <code>notebooks/models</code> to project-root paths
2	Empty live CSV → NaN in fetch-since logic	Data Collection / Ingestion	Guarded with a non-empty check before taking <code>max(date)</code>
3	Pandas dtype upcast on concat	Data Collection / Ingestion	Excluded empty frames from the concat so dtypes infer correctly
5	ATR feature ~45,000x scale mismatch	Feature Engineering	Added <code>atr_pct = atr_14 / Close</code> ; R^2 went from -0.65

#	Issue	Pipeline Stage	Resolution
7	$R^2 \approx 0.99$ was a target-overlap artifact	Feature Engineering	to +0.18 Set forecast horizon = rolling window (168h), eliminating the overlap
4	<code>ewma_sigma()</code> silently returned all NaN	Modelling	Seeded the recursion with <code>np.nanmean()</code> instead of <code>.mean()</code>
6	GARCH forecast was effectively static	Modelling	Rebuilt as a genuine rolling multi-step <code>.forecast(horizon=H)</code> call
8	LSTM had no econometric grounding	Modelling	Replaced with EWMA (RiskMetrics), later re-tuned over a wider λ grid
7a	Live-test window shrank to ≈ 48 hours	Live Testing / Evaluation	Switched to fixed calendar boundaries, widening the window to ~ 8 months
9	“Beats persistence?” values went stale	Live Testing / Evaluation	Recomputed against the corrected 168h RMSE table
10	Double-escaped <code>\n\n</code> broke report paragraphs	Report / Deck Generation	Fixed the raw JSON escaping and regenerated the docx/PDF
11	python-pptx multi-paragraph shape corruption	Report / Deck Generation	Edited/cloned paragraph XML directly; added visual review of every slide

No EDA-stage issues appear below. EDA in this project is visual and exploratory (distribution plots, ACF/PACF, correlation matrices, boxplots) — nothing computed there feeds silently into a later stage the way a feature, a model, or a live-scoring pipeline does, so there was nothing for a bug to hide inside. Every issue below instead clusters at a boundary between two stages: where one stage’s output became the next stage’s untrusted input (Data Collection → Feature Engineering, Feature Engineering → Modelling, Modelling → Live Testing, analysis → report/deck). See Lessons Learned.

A. Data Collection / Ingestion

1. Notebook execution working-directory mismatch

Problem: Running the notebook headlessly via `jupyter nbconvert --to notebook --execute --inplace` starts the kernel with its working directory set to the notebook's own folder (`notebooks/`), not the project root. Relative paths like `data/btc_hourly_ohlcv.csv` and `models/*.pkl`, which worked fine in an interactive Jupyter session launched from the project root, failed to resolve during automated execution.

Fix: Created symlinks `notebooks/data → ../data` and `notebooks/models → ../models`, so the same relative paths resolve correctly regardless of which directory the kernel treats as its working directory.

2. Empty live-test CSV producing NaN in the fetch-since logic

Problem: On the very first run, before any live-test data had been cached, the code that determines “fetch new candles since the last cached timestamp” read an empty CSV. Calling `.max()` on an empty date column returned NaN, and that NaN was then passed to the exchange API as the fetch start time.

Fix: Added a `len(df_live_existing) > 0` guard before computing the max cached timestamp, falling back to a full historical fetch whenever no cache exists yet.

3. Pandas silently upcast every column to object dtype on concat

Problem: Concatenating an empty, CSV-loaded DataFrame (which pandas types every column as `object` when there are zero rows to infer a numeric type from) with a freshly fetched, correctly float-typed DataFrame silently upcasts the combined result's columns to `object` dtype. No error is raised — the corruption only shows up later, as numeric operations that should work start failing or behaving strangely.

Fix: Guarded the concatenation to only include non-empty frames, so dtype inference always runs against real, populated data.

B. EDA

No issues to report at this stage — see the note under the Summary Table for why.

C. Feature Engineering

5. ATR regressor was on the wrong scale by a factor of $\approx 45,000\times$

Problem: HAR-RV-X's regressors are all meant to sit on a comparable, fractional scale (roughly 0.001–0.03). `atr_14` was left in raw dollar terms instead — mean $\approx \$303$, ranging up to $\approx \$3,189$ across BTC's price history. Worse, because BTC's own price ranged from about \$4,000 to over \$90,000 across the dataset, a fixed dollar ATR value meant something completely different depending on when in history it was measured, so the Ridge regression had no stable relationship to learn from that column.

Fix: Engineered `atr_pct = atr_14 / Close`, putting ATR on the same price-normalized fractional scale as every other HAR-RV-X regressor.

Impact: This was the single highest-impact fix in the project. It took HAR-RV-X's historical R^2 from -0.65 to $+0.18$, turning it from a broken feature into the strongest model in the whole comparison, on both the historical and live-test splits. It was found while systematically testing modeling-improvement ideas — the only one of six tested ideas that turned out to be a genuine bug rather than a modeling choice.

7. Suspiciously high R^2 (≈ 0.99) traced to a target-construction artifact

Problem: At the project's original 1-hour-ahead forecast horizon, even the naive-persistence and Holt-Winters baselines scored an R^2 near 0.99 — too good to be genuine forecasting skill. Investigation traced the cause to how the target was built: a 168-hour rolling-window volatility figure, sampled every 1 hour, shares 167 of its 168 underlying hourly returns (99.4%) with its own value from one hour earlier. “Forecasting” it one hour ahead was therefore close to just copying the previous known value, not a real prediction.

Fix: The user proposed, and it was implemented, setting the forecast horizon equal to the rolling window length itself (168 hours). At that horizon, the forecast origin's most recently known volatility value and the target share zero overlapping return observations — the standard non-overlapping design used in the realized-volatility literature.

Impact: Fixing this changed the entire model ranking: naive persistence and Holt-Winters, which had dominated under the flawed design, now correctly post negative R^2 , while HAR-RV-X and EWMA emerge as the genuinely working models. This is a target-construction issue, not a model-fitting one — it belongs here, upstream of any model actually being trained on the corrected target.

D. Modelling

4. `ewma_sigma()` silently returned NaN for every live-test prediction

Problem: The recursive EWMA volatility function seeded its running estimate using a plain `.mean()` call. `combined_features['hourly_return']` has one structural leading NaN —

the very first row in the series has no prior price to compute a return from — and because `.mean()` is not NaN-safe, that single NaN silently propagated through the entire recursive computation. Every downstream EWMA prediction on the live-test set came out as NaN, with no exception thrown anywhere.

Fix: Replaced the seeding call with `np.nanmean()`, which skips the NaN and seeds the recursion from the first valid return.

Filed under Modelling, not Data Collection: the buggy function is the EWMA model's own recursive estimator, not a data-fetching or cleaning step — it only manifested during live scoring because that was the first time the recursion ran over freshly-built, not-yet-training-cleaned features.

6. GARCH's multi-step forecast was effectively static

Problem: The original GARCH forecasting code did not perform a genuine rolling multi-step-ahead forecast across the test set — the forecast did not properly roll forward origin-by-origin the way a real out-of-sample evaluation requires.

Fix: Rebuilt the forecasting loop around `.fix(params)` with a rolling `horizon=1` forecast; later, as part of the 168-hour horizon redesign, upgraded it to a genuine multi-step call — `.forecast(horizon=HORIZON, start=date, reindex=False)` — taking column `[:, -1]` (the array's shape is `(n_origins, horizon)`) as the H-step-ahead forecast for each origin.

8. LSTM had no econometric grounding for this problem

Problem: The project's original third model slot was an LSTM. On reflection, a generic recurrent sequence model has no built-in notion of volatility, risk, or econometric structure — it just learns an opaque nonlinear mapping over raw sequences, which sits awkwardly in a project framed as a quantitative time-series / risk-management study alongside GARCH and HAR-RV.

Fix: Replaced it with EWMA (exponentially weighted moving average volatility, the RiskMetrics-style estimator actually used on trading-desk risk systems) — a real risk-management tool with theoretical grounding, not a black box. Its decay factor λ was later re-tuned over a wider search grid (0.94 default → 0.995 tuned) as one of the systematic model-improvement passes.

E. Live Testing / Evaluation

7a. Consequence of the horizon fix: the live-test window shrank to ≈ 48 hours

Problem: A prediction made 168 hours ahead of its features cannot be scored until 168 real hours have actually elapsed. With the live-test period still anchored to the original short, rolling “recent days” window, only a couple of days’ worth of predictions were old enough to have a real outcome to compare against — far too few to evaluate live performance meaningfully.

Fix: The user proposed switching from a rolling “N years back from today” date scheme to fixed calendar boundaries: training data spanning 2010-01-01 (Bitcoin’s own origin) through 2026-01-01, with the live-test window running from 2026-01-02 through yesterday. This expanded the live-test window to roughly eight months.

Impact: 5,664 live-test predictions are now logged, with 5,496 of them already old enough to be scored as of the last full run.

9. “Beats persistence?” comparison values went stale after the horizon redesign

Problem: Comparison tables in the report and slide deck included a “beats naive persistence?” column computed once, early in the project. Under the original flawed 1-hour design, only Holt-Winters appeared to beat the naive baseline — itself an artifact of the same overlap problem described in Issue 7. Once the horizon redesign changed the actual model rankings, this column no longer matched the new numbers.

Fix: Recomputed every “beats persistence?” value directly against the corrected 168-hour RMSE table before propagating the numbers into the report and slide deck. Under the corrected design, HAR-RV-X and EWMA both genuinely beat naive persistence; naive persistence and Holt-Winters do not beat themselves, by definition, and GARCH does not beat persistence either.

F. Report / Deck Generation

These issues were in the tooling used to produce the deliverables (Word report, PDF, and PowerPoint deck) rather than in the underlying analysis — but they were just as capable of silently shipping wrong content, so they’re recorded with the same rigor.

10. Double-escaped `\n\n` broke paragraph breaks in the generated report

Problem: `report_results.json`’s long-form narrative fields (`conclusion_text`, `live_test_discussion`) contained a literal four-backslash sequence (`\\n\\n`) in the raw file instead of a proper JSON `\n\n` escape sequence. When substituted into the Markdown report template, this rendered as literal text rather than a paragraph break, causing sentences to run together mid-word — for example, “...more interesting.headline finding is...” with no space or break at all.

Fix: Patched by replacing the raw four-backslash sequence with a proper JSON-escaped newline directly in the file text (before JSON parsing), verified the fix by reloading with `json.load()`, and regenerated both the `.docx` and `.pdf` report.

11. `python-pptx` multi-paragraph shape corruption (hit multiple times)

Problem: Some PowerPoint text-box shapes internally contain several separate `<a:p>` paragraph elements, even though `python-pptx`’s convenient `.text` property shows them joined together with `\n` characters when read back. A text-replacement helper that follows the common single-paragraph pattern — setting only `paragraphs[0].runs[0].text` — silently stuffs an entire multi-line replacement string into paragraph 0 while leaving the shape’s original

paragraphs 1, 2, etc. completely untouched. The result is duplicated or visually overflowing text on the rendered slide, and nothing about the python-pptx API call itself raises an error or warning. A related variant hit later in the project: a shape's replacement text ran long enough to visually overflow its container even with the paragraph structure correct, and separately, a shape's "old" and "new" text were briefly concatenated across two runs (`run[0]/run[1]`) that were assumed to be a bold-label/body split but were actually the full old text and full new text stacked back to back.

Fix: On the "What one record is" bullet list (slide 4): restored paragraph 0's correct short text, then cloned paragraph 2's underlying XML element (via a deep copy inserted immediately after it) to properly add a new fourth bullet as its own paragraph, rather than concatenating more text into a single paragraph. On the "History" stat card (slide 4): set `paragraphs[0]` and `paragraphs[1]` as two distinct paragraph objects instead of embedding a `\n` inside one `set_text()` call. Later instances were fixed by explicitly inspecting `len(shape.text_frame.paragraphs[i].runs)` before any blind text overwrite, and by adding new slides through proper slide duplication (the pptx skill's `add_slide.py`) rather than building layouts from scratch.

Impact: Adopted a standing practice for the rest of the deck build: render every edited slide to an image (via a PDF export and `pdftoppm`) and visually inspect it before considering an edit finished. This is what caught every instance of this bug, and every one of the deck's other edited slides was subsequently confirmed clean the same way before delivery.

Lessons Learned

- **Silent failures were the most dangerous category by far.** NaN propagation, dtype upcasting, and multi-paragraph shape corruption never raised an exception — every one of them was only caught through a downstream sanity check or a manual visual inspection, not an error message.
- **Every silent-failure bug in this log sits at a boundary between two pipeline stages, not inside one.** Grouping issues by stage (above) makes this visible directly: Data Collection → Feature Engineering (Issues 2, 3), Feature Engineering → Modelling (Issue 5), Modelling → Live Testing (Issue 4), and analysis → deliverable (Issues 10, 11). Nothing broke silently in the middle of a single stage's own logic — it broke exactly where one stage's output became the next stage's untrusted input. EDA had zero issues for the same reason in reverse: it's a dead end, not a boundary — nothing downstream silently trusts what EDA produces.
- **A metric that looks too good deserves at least as much scrutiny as one that looks bad.** The $R^2 \approx 0.99$ result (Issue 7) was a bug; the $R^2 \approx -11$ result for GARCH later in the project was real, structural, and correctly reported as such.
- **Feature scale mismatches can silently cripple a linear model without any error at all** (Issue 5) — catching it required actually reasoning about the numeric ranges involved, not just checking that the code ran.
- **The most important methodological finding of the whole project may be a negative result.** A cross-validated check on an ensemble of HAR-RV-X and EWMA — which

looked like a genuine improvement on a naive test-set comparison — revealed the CV-selected blend weight converged to 100% HAR-RV-X. The apparent improvement was noise from tuning against the test set, the same category of mistake as Issue 7, caught the second time only because rolling-origin cross-validation was applied rigorously rather than skipped as “obviously fine.”

- **Regenerating downstream deliverables (report, deck) after a substantive analysis change is its own source of bugs**, distinct from the analysis itself — tooling gotchas like Issues 10 and 11 are easy to miss precisely because the underlying numbers are already correct by that point.